

AI Engineering

A Comprehensive Guide

From Foundations to Fine-Tuning

Prepared for: Tirth | Capstone Project & Interview Preparation
April 2026

Table of Contents

Chapter 1: Large Language Models — The Foundation

Chapter 2: Embeddings & Vector Spaces

Chapter 3: Prompt Engineering

Chapter 4: Synthetic Data Generation

Chapter 5: Fine-Tuning & LoRA

Chapter 6: Retrieval-Augmented Generation (RAG)

Chapter 7: Evaluation Frameworks

Chapter 8: Your Capstone Pipeline

Chapter 9: Interview Quick Reference

Chapter 10: Complete Reading & Resource List

Chapter 1: Large Language Models — The Foundation

Large Language Models (LLMs) are neural networks trained on massive text datasets to predict the next token in a sequence. They form the backbone of modern AI applications.

1.1 What is a Language Model?

At its core, a language model answers one question: given some text, what comes next? It assigns probabilities to possible next tokens.

***Analogy:** Think of autocomplete on your phone, but scaled to billions of parameters. Your phone predicts the next word based on patterns. An LLM does the same thing, but with so much training data that it can write essays, code, and have conversations.*

1.2 Tokens & Tokenization

LLMs don't read words — they read tokens. A tokenizer splits text into subword units the model can process.

- **Token:** A subword unit. "Hello world" might be 2 tokens. "Unbelievable" might be 3: [Un][believ][able].
- **Vocabulary:** The set of all tokens the model knows (typically 32K–128K tokens).
- **BPE (Byte-Pair Encoding):** The most common tokenization algorithm. Starts with characters, iteratively merges frequent pairs.
- **Context Window:** Maximum tokens processed at once. Claude Haiku: 200K. Llama 3: 8K–128K.

***Interview Tip:** If asked "What is a token?", mention that token counts affect cost (API pricing is per-token), latency, and context limits. Show you understand practical implications.*

1.3 The Transformer Architecture

The Transformer (Vaswani et al., 2017) is the architecture behind all modern LLMs.

Self-Attention Mechanism

The core innovation. For each token, attention computes how much every other token is relevant to it.

- **Query (Q):** What am I looking for?
- **Key (K):** What do I contain?
- **Value (V):** What information do I provide?

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

Analogy: Imagine reading a sentence. When you read “it”, your brain automatically looks back to figure out what “it” refers to. Self-attention is the model doing exactly this — for every word, simultaneously.

Multi-Head Attention

Instead of one attention computation, the model runs several in parallel (heads), each learning different relationships (syntax, semantics, coreference, etc.).

Feed-Forward Networks

After attention, each token passes through a small neural network independently. This is where factual knowledge is believed to be stored.

Layer Normalization & Residual Connections

These stabilize training. Residual connections allow gradients to flow through deep networks (LLMs have 32–80+ layers).

Q: Explain the Transformer architecture in 2 minutes.

A: A Transformer processes all tokens in parallel using self-attention, which lets each token attend to every other token. It has an encoder-decoder structure (for translation) or decoder-only (for generation, like GPT/Claude). Each layer has multi-head self-attention (captures different relationships), a feed-forward network (stores knowledge), and residual connections with layer normalization for stable training. The key innovation over RNNs is parallelization — no sequential processing bottleneck.

1.4 Pre-Training vs. Inference

Aspect	Pre-Training	Inference
Goal	Learn language patterns from data	Generate responses to user input
Data	Terabytes of text (web, books, code)	Your specific prompt/conversation
Cost	Millions of dollars, weeks on GPUs	Cents per request
Who does it	Model providers (Anthropic, Meta)	You, via API or local deployment
Output	Model weights (parameters)	Generated text

Key takeaway: You don’t pre-train models. You use pre-trained models and either prompt them or fine-tune them.

1.5 Key Models You Should Know

Model	Creator	Open/Closed	Key Feature
GPT-4/4o	OpenAI	Closed (API)	Strong general reasoning

Model	Creator	Open/Closed	Key Feature
Claude (Haiku/Sonnet/Opus)	Anthropic	Closed (API)	Long context, instruction following
Llama 3 (8B/70B)	Meta	Open weights	Best open-source, fine-tunable
Gemma 2 (2B/9B/27B)	Google	Open weights	Lightweight, efficient
Mistral/Mixtral	Mistral AI	Open weights	Good performance/size ratio
Qwen 2.5	Alibaba	Open weights	Strong multilingual

For your project: You're using Claude Haiku (closed, via API) as your prompted baseline, and will fine-tune Llama or Gemma (open) as your specialist model.

1.6 Recommended Reading

- **Video:** Andrej Karpathy — “Intro to Large Language Models” (YouTube, 1hr)
- **Video:** 3Blue1Brown — “But what is a GPT?” series
- **Paper:** “Attention Is All You Need” (Vaswani et al., 2017)
- **Blog:** Jay Alammar — “The Illustrated Transformer”

Chapter 2: Embeddings & Vector Spaces

Embeddings are how machines understand meaning. They convert text into numerical vectors where similar meanings are close together in space.

2.1 What Are Embeddings?

An embedding is a dense vector (list of numbers) that represents the meaning of text. Typically 384–4096 dimensions.

Analogy: *Imagine a map where cities are placed by similarity, not geography. Paris and Rome would be close (both European capitals), while Paris and a random village would be far. Embeddings create this kind of map, but for text meaning.*

- **Word embeddings:** One vector per word. “king” – “man” + “woman” $\approx$ “queen”
- **Sentence embeddings:** One vector for the whole sentence. Used in RAG for retrieval.
- **Cosine similarity:** Measures how similar two vectors are (1.0 = identical, 0 = orthogonal, -1.0 = opposite)

2.2 Why Embeddings Matter for Your Project

In RAG, when a client says “I feel like giving up,” you need to find relevant knowledge base documents. Keyword search would miss “hopelessness” or “despair.” Embedding-based search finds semantically similar content regardless of exact words.

2.3 Popular Embedding Models

Model	Dimensions	Use Case
OpenAI text-embedding-3-small	1536	General purpose, API-based
all-MiniLM-L6-v2	384	Lightweight, local, fast
BAAI/bge-large-en-v1.5	1024	High quality, local
Cohere embed-v3	1024	Multilingual, API-based

Q: How would you choose an embedding model?

A: Consider: (1) Dimensionality — higher dims capture more nuance but cost more to store/search. (2) Domain match — models trained on similar data perform better. (3) Speed vs quality tradeoff. (4) Local vs API — local avoids vendor lock-in. For a therapy chatbot, a local model like bge-large balances quality and control.

2.4 Recommended Reading

- **Blog:** Jay Alammar — “The Illustrated Word2Vec”
- **Course:** Stanford CS224N — Lecture 1 on word vectors

Chapter 3: Prompt Engineering

Prompt engineering is the art and science of crafting inputs that get the best outputs from LLMs. It's your primary tool before fine-tuning.

3.1 Types of Prompts

System Prompt

Sets the model's role, personality, and behavioral constraints. This is what you've built for your MI therapist chatbot.

Few-Shot Prompting

Providing examples in the prompt so the model follows a pattern. Give 2–5 examples. More isn't always better — it uses up context window.

Chain-of-Thought (CoT)

Asking the model to reason step-by-step before answering. Dramatically improves reasoning tasks.

Zero-Shot vs Few-Shot

Approach	Description	When to Use
Zero-shot	No examples, just instructions	Simple tasks, capable models
One-shot	One example	When format matters
Few-shot	2–5 examples	Complex or nuanced tasks

3.2 Key Parameters

- **Temperature (0–1):** Controls randomness. 0 = deterministic, 1 = creative. For therapy: 0.3–0.7.
- **Top-p (nucleus sampling):** Only consider tokens whose cumulative probability reaches p. Top-p=0.9 means the smallest set covering 90% probability.
- **Max tokens:** Maximum response length. Set appropriately to avoid cut-off or wasted cost.
- **Stop sequences:** Tokens that tell the model to stop generating.

***Interview Tip:** Know the difference between temperature and top-p. Temperature scales the logits (pre-softmax scores); top-p truncates the distribution. Most practitioners use one or the other, not both.*

3.3 Advanced Techniques

- **Structured output:** Ask for JSON or specific formats. Use schema descriptions.

- **Role prompting:** Assign a specific expert role for domain-specific responses.
- **Self-consistency:** Generate multiple answers and take the majority vote.
- **ReAct:** Interleave reasoning and actions (think → act → observe → think).

3.4 Recommended Reading

- **Guide:** Anthropic Prompt Engineering Guide (docs.anthropic.com)
- **Paper:** “Chain-of-Thought Prompting Elicits Reasoning” (Wei et al., 2022)
- **Paper:** “ReAct: Synergizing Reasoning and Acting” (Yao et al., 2022)

Chapter 4: Synthetic Data Generation

Synthetic data is artificially generated training data, typically created by a stronger model to train a weaker one. One of the most important techniques in modern AI engineering.

4.1 Why Synthetic Data?

- **Scarcity:** Real therapy conversations are rare, sensitive, and expensive to label.
- **Control:** You define the labels (emotion, defensiveness) — no manual annotation needed.
- **Diversity:** Systematically cover edge cases that rarely appear in real data.
- **Cost:** Dramatically cheaper than hiring human annotators.
- **Privacy:** No real patient data involved.

4.2 The Teacher-Student Paradigm

The core idea: a large, capable model (teacher) generates training data for a smaller model (student).

Role	In Your Project	Purpose
Teacher	Claude Sonnet/Opus	Generates 1000 therapy conversations with labels
Student	Llama 3 / Gemma 2	Fine-tuned on synthetic data to become specialist
Evaluator	Claude Opus + AnnoMI	Judges the student's quality

***Analogy:** It's like a master chef (Claude) writing down 1000 recipes with detailed notes, then using those recipes to train an apprentice (Llama) who eventually becomes specialized in one cuisine.*

4.3 Key Principles

Diversity is Critical

Your 270,000 unique scenario combinations prevent the model from learning narrow patterns.

- **Client profiles:** 25 different personalities, defensiveness levels, backgrounds
- **Emotion trajectories:** 15 different emotional arcs
- **Session contexts:** 10 different therapeutic settings
- **Therapist styles:** 6 approaches to prevent stylistic overfitting

Label Quality

Synthetic labels are only as good as the teacher model's understanding. Validate a sample against human judgment.

Data Contamination

Critical concept: Training data and evaluation data **MUST** be independent. This is why you use AnnoMI (real) for evaluation and synthetic data for training.

4.4 Common Pitfalls

- **Mode collapse:** All generated conversations sound the same if prompts aren't diverse enough.
- **Hallucinated labels:** Teacher might mislabel defensiveness levels.
- **Distribution imbalance:** 90% low-defensiveness data hurts performance on hard cases.

Q: What are the risks of training on synthetic data?

A: Three main risks: (1) Distribution shift — synthetic data may not match real-world patterns. (2) Error amplification — teacher model biases get baked into the student. (3) Homogeneity — synthetic conversations can lack real human messiness. Mitigations: validate against real data (AnnoMI), maximize scenario diversity, human-spot-check a sample.

4.5 Landmark Work

- **Stanford Alpaca (2023):** Pioneered using GPT to generate 52K examples. Fine-tuned Llama 7B to match GPT-3.5.
- **Self-Instruct (2022):** Framework for generating instruction data from a seed set.
- **Orca (Microsoft, 2023):** Used explanation traces in synthetic data — student learned reasoning, not just answers.

4.6 Recommended Reading

- **Paper:** “Self-Instruct” (Wang et al., 2022)
- **Paper:** “Orca: Progressive Learning from Complex Explanation Traces” (Mukherjee et al., 2023)
- **Blog:** Stanford Alpaca blog post (crfm.stanford.edu)

Chapter 5: Fine-Tuning & LoRA

Fine-tuning takes a pre-trained model and trains it further on your specific data. LoRA makes this feasible on consumer hardware.

5.1 Why Fine-Tune?

Approach	Pros	Cons
Prompting only	No training needed, flexible, easy to iterate	Limited by context window, inconsistent, expensive at scale
Fine-tuning	Consistent behavior, lower inference cost, specialized	Requires data and compute, less flexible
Both combined	Best of both worlds	More complex pipeline

For your project: Compare prompting-only (Claude Haiku + MI system prompt) vs fine-tuned (LoRA-adapted Llama/Gemma on 1000 conversations).

5.2 Full Fine-Tuning vs. PEFT

Full Fine-Tuning

Updates ALL model parameters. For 7B parameters: ~28GB GPU memory just for weights, often 3–4x total with optimizer states. Impractical for most people.

LoRA (Low-Rank Adaptation)

The breakthrough technique published by Hu et al. (2021) at Microsoft.

Core idea: Freeze the original model and inject small trainable “adapter” matrices into specific layers.

***Analogy:** Imagine a textbook (pre-trained model). Full fine-tuning rewrites the entire book. LoRA staples in a few pages of notes at key chapters. The notes are tiny compared to the book, but they change how you interpret the content.*

How LoRA Works

For a weight matrix W of size $d \times d$ (say 4096×4096):

- **Original:** $\text{output} = W \times \text{input}$ (16.7M parameters)
- **LoRA:** $\text{output} = (W + BA) \times \text{input}$, where B is $d \times r$ and A is $r \times d$
- **With $r=16$:** B is 4096×16 , A is 16×4096 = only 131K params (0.8% of original!)

Only A and B are trained. W stays frozen. At inference, merge $W+BA$ — zero overhead.

5.3 Key Hyperparameters

Parameter	What It Controls	Typical Values	Your Project
Rank (r)	Adapter capacity	4, 8, 16, 32, 64	Start with r=16
Alpha (α)	Scaling factor	16, 32, 64	Alpha = 32
Target modules	Which layers get adapters	q_proj, v_proj, k_proj	q_proj, v_proj
Learning rate	How fast weights change	1e-4 to 3e-4	2e-4
Epochs	Passes through dataset	1–5	3
Batch size	Examples per update	4–32	8
Warmup steps	Gradual LR increase	5–10% of total	10%

5.4 QLoRA — Even More Efficient

QLoRA (Dettmers et al., 2023) combines LoRA with 4-bit quantization:

- **Quantization:** Compresses weights from 16-bit to 4-bit, reducing memory ~4x
- **NF4:** A special 4-bit data type optimized for normally-distributed weights
- **Result:** Fine-tune 7B model on a single 24GB GPU (RTX 3090/4090)

Q: Explain LoRA and why it works.

A: LoRA hypothesizes that weight changes during fine-tuning have low intrinsic rank — you don't need to update millions of parameters. It injects pairs of small matrices (A and B) into transformer layers; only these are trained while original weights stay frozen. A rank-16 LoRA on 7B trains only ~0.1% of parameters. It works because task-specific adaptations are simpler than general language understanding — pre-trained weights already capture language; LoRA just nudges behavior.

5.5 Training Data Format

For instruction fine-tuning, data is formatted as conversations:

```
{"messages": [{"role": "system", "content": "You are an MI therapist..."},
{"role": "user", "content": "I don't think I have a problem."},
{"role": "assistant", "content": "It sounds like..."}]}
```

5.6 The Fine-Tuning Stack

Tool	Purpose
Hugging Face Transformers	Load and run models
PEFT	LoRA implementation

Tool	Purpose
TRL (SFTTrainer)	Supervised fine-tuning
BitsAndBytes	4-bit quantization for QLoRA
Weights & Biases	Track training metrics

5.7 Recommended Reading

- **Paper:** “LoRA: Low-Rank Adaptation” (Hu et al., 2021)
- **Paper:** “QLoRA” (Dettmers et al., 2023)
- **Guide:** Sebastian Raschka — “Practical Tips for Finetuning LLMs”
- **Tutorial:** Hugging Face PEFT docs + SFTTrainer tutorial

Chapter 6: Retrieval-Augmented Generation (RAG)

RAG combines retrieval (finding relevant documents) with generation (LLM producing responses). It gives the model access to knowledge not in its training data.

6.1 Why RAG?

- **Knowledge cutoff:** The model only knows its training data. Can't access new guidelines.
- **Hallucination:** Without grounding, models confidently make things up. RAG provides factual anchors.

For your project: Knowledge base of MI techniques. RAG retrieves relevant techniques based on what the client says, grounding therapist responses in real methodology.

6.2 The RAG Pipeline

Phase 1: Indexing (Offline)

- 1. **Load documents:** Read knowledge base files
- 2. **Chunk:** Split into smaller pieces (256–1024 tokens each)
- 3. **Embed:** Convert each chunk into a vector
- 4. **Store:** Save vectors in a vector database

Phase 2: Retrieval + Generation (Online)

- 1. **Embed query:** Convert user message into a vector
- 2. **Search:** Find top-k most similar chunks
- 3. **Augment prompt:** Insert retrieved chunks as context
- 4. **Generate:** LLM produces grounded response

6.3 Chunking Strategies

Strategy	How It Works	Best For
Fixed-size	Split every N tokens with overlap	Simple, works for most cases
Sentence-based	Split at sentence boundaries	Self-contained sentences
Semantic	Split when topic changes	Documents with clear topic shifts
Recursive	Try large splits, subdivide if too big	Default in LangChain

- **Chunk size:** Start with 512 tokens. Too small loses context; too large adds noise.
- **Overlap:** 10–20% between chunks prevents cutting context at boundaries.

6.4 Vector Stores

Store	Type	Best For
ChromaDB	Local, embedded	Prototyping, small datasets (your project)
FAISS	Local, Facebook AI	Fast similarity search, medium datasets
Pinecone	Cloud, managed	Production, large scale
Weaviate	Cloud/local	Hybrid search (vector + keyword)

6.5 Advanced RAG Techniques

- **Hybrid search:** Combine vector similarity with keyword search (BM25).
- **Reranking:** Use cross-encoder to re-score initial retrieval results.
- **Query transformation:** Rewrite queries to be more retrieval-friendly.
- **Multi-query:** Generate query variations and merge results.
- **Contextual compression:** Extract only relevant parts of retrieved chunks.

Q: How would you improve a RAG system returning irrelevant results?

A: Diagnose systematically: (1) Check chunking. (2) Try hybrid search (vector + BM25). (3) Add a reranker. (4) Query transformation. (5) Check embedding model domain fit. (6) Evaluate retrieval separately — measure recall@k before blaming the LLM.

6.6 Recommended Reading

- **Tutorial:** LangChain RAG tutorial
- **Blog:** Pinecone — “What is RAG?”
- **Paper:** “Retrieval-Augmented Generation for Knowledge-Intensive NLP” (Lewis et al., 2020)

Chapter 7: Evaluation Frameworks

This is the most important skill in AI engineering. Anyone can fine-tune a model. The hard part is proving it actually works.

7.1 Types of Evaluation

Type	What It Measures	In Your Project
Automated metrics	Statistical text similarity	BLEU/ROUGE vs gold-standard
LLM-as-judge	Quality rated by another LLM	Claude Opus rates empathy, MI adherence
Human evaluation	Expert ratings	Therapist reviews sample responses
Behavioral tests	Specific capability checks	Identifies high defensiveness?
A/B comparison	Side-by-side preference	Prompted vs fine-tuned

7.2 Automated Metrics

BLEU

Measures n-gram overlap between generated and reference text. Scores 0–1. Good for terminology checks, bad for creative/open-ended tasks.

ROUGE

Focuses on recall (how much reference is captured). ROUGE-1 = unigram overlap, ROUGE-L = longest common subsequence.

BERTScore

Uses embeddings instead of exact matching. More forgiving of paraphrasing. Better for open-ended generation.

Interview Tip: Acknowledge that automated metrics are limited for conversational AI. The real evaluation is LLM-as-judge and human evaluation. Automated metrics are a quick sanity check.

7.3 LLM-as-Judge

Using a strong model (Claude Opus) to evaluate your fine-tuned model's outputs. Most practical evaluation method.

- **Define a rubric:** Specific criteria with clear scoring guidelines
- **Structured prompt:** Give judge the conversation, response, and rubric
- **Multiple dimensions:** Empathy (1–5), MI technique accuracy (1–5), naturalness (1–5)

- **Include reasoning:** Ask judge to explain score before giving number (CoT)

Example Rubric for Your Project

Dimension	1 (Poor)	5 (Excellent)
MI Adherence	Confrontational, unsolicited advice	Uses OARS consistently, rolls with resistance
Empathy	Dismissive, ignores emotions	Deeply reflective, validates accurately
Defensiveness Handling	Escalates resistance	De-escalates skillfully
Naturalness	Robotic, formulaic	Sounds like a real therapist
Safety	Gives harmful advice	Appropriate boundaries, suggests help

7.4 Behavioral Testing

Write specific test cases for expected behaviors:

- **Suicidal ideation** → Model recommends crisis resources
- **Highly defensive client** → Model does NOT confront or argue
- **Readiness to change** → Model reinforces change talk

Q: How do you evaluate an LLM-based application?

A: Multi-layer approach: (1) Behavioral tests for safety — pass/fail. (2) LLM-as-judge with rubric for quality dimensions. (3) Automated metrics (BLEU, BERTScore) as regression checks. (4) Human evaluation on a sample for ground truth. Track all metrics over time. Keep eval data strictly separate from training data.

7.5 Recommended Reading

- **Blog:** Hamel Husain — “Your AI Product Needs Evals”
- **Paper:** “Judging LLM-as-a-Judge” (Zheng et al., 2023)

Chapter 8: Your Capstone Pipeline

How all the concepts connect in your specific project:

8.1 Architecture Overview

Stage	Concept	What You're Doing	Tools
1. Data	Real data for eval	AnnoMI (133 conversations)	Python, Kaggle
2. Synthetic	Teacher-student	Claude generates 1000 MI convos	Claude CLI
3. Process	Training format	Convert JSONL to chat format	Python
4. Fine-tune	LoRA/QLoRA	Adapt Llama/Gemma	HF, PEFT, TRL
5. RAG	Retrieval-augmented	MI techniques knowledge base	LangChain, Chroma
6. Evaluate	Multi-dimensional	Prompted vs fine-tuned	LLM-as-judge
7. Demo	End-to-end app	Streamlit chatbot	Streamlit

8.2 What Makes This Impressive

- **Data engineering:** Synthetic data pipeline with controlled diversity
- **Model development:** Fine-tuning with LoRA, not just prompt engineering
- **RAG integration:** Grounding responses in domain knowledge
- **Rigorous evaluation:** Multi-dimensional comparison with real benchmarks
- **Domain specialization:** Applied to meaningful use case (mental health)

This is exactly the kind of end-to-end project AI engineering hiring managers look for.

8.3 Interview Framing

- **Trade-offs:** “I chose LoRA over full fine-tuning because...” — shows engineering judgment
- **Evaluation rigor:** “I used LLM-as-judge with a 5-dimension rubric AND behavioral tests”
- **Data quality:** “270K unique scenario combinations to prevent mode collapse”
- **Real vs synthetic:** “Kept evaluation on real data (AnnoMI) to avoid contamination”

Chapter 9: Interview Quick Reference

Common AI engineering interview questions with concise answers.

9.1 Fundamentals

Q: What's the difference between a language model and a chatbot?

A: A language model is the engine — it predicts next tokens. A chatbot is an application built on top, adding conversation management, system prompts, memory, and UI.

Q: What is the attention mechanism?

A: Attention lets each token look at every other token to determine relevance. It computes Query-Key similarity scores, normalizes with softmax, and weights Value vectors. Captures long-range dependencies that RNNs struggle with.

Q: Encoder vs decoder models?

A: Encoders (BERT) process input bidirectionally — good for classification. Decoders (GPT, Claude, Llama) generate left-to-right — good for generation. Most modern LLMs are decoder-only.

9.2 Fine-Tuning & Training

Q: When would you fine-tune vs. just prompt?

A: Prompt when: quick iteration, general task, limited data, behavior changes often. Fine-tune when: consistent specialized behavior needed, quality training data available, lower inference cost matters.

Q: What is catastrophic forgetting?

A: When fine-tuning causes the model to lose pre-trained capabilities. Training too aggressively on therapy data might break basic reasoning. LoRA mitigates this because original weights are frozen.

Q: Difference between SFT, RLHF, and DPO?

A: SFT: train on input-output pairs. RLHF: train reward model on human preferences, optimize LLM against it. DPO: skip reward model, directly optimize on preference pairs. SFT is simpler; RLHF/DPO are for alignment.

9.3 RAG & Retrieval

Q: RAG vs fine-tuning?

A: RAG for: factual accuracy, up-to-date info, traceable sources. Fine-tuning for: behavioral changes, consistent style, domain reasoning. They're complementary.

Q: Dense vs sparse retrieval?

A: Sparse (BM25): keyword matching, fast, no ML. Dense (embeddings): semantic similarity, catches paraphrases. Hybrid is usually best.

9.4 Evaluation & Production

Q: How do you detect hallucinations?

A: (1) Groundedness checks vs retrieved docs. (2) Self-consistency across multiple generations. (3) Confidence calibration — flag low-confidence outputs.

Q: Main challenges deploying LLMs in production?

A: Latency, cost, safety, evaluation at scale, context management, and reliability. Prioritize evaluation and safety guardrails first.

9.5 Concepts to Name-Drop

Concept	One-Liner
RLHF	RL from human preferences — how Claude/GPT get aligned
DPO	Direct preference optimization — simpler RLHF alternative
Mixture of Experts	Activate subset of params per input — efficient scaling
Quantization	Compress weights (32-bit → 4-bit) for cheaper inference
Distillation	Train small model to mimic large one
Constitutional AI	Anthropic's approach — model self-critiques against principles
Agentic AI	LLMs that use tools, plan, and act autonomously

Chapter 10: Complete Reading & Resource List

10.1 Must-Read Papers

- “Attention Is All You Need” (Vaswani et al., 2017) — The Transformer
- “LoRA: Low-Rank Adaptation” (Hu et al., 2021)
- “QLoRA” (Dettmers et al., 2023) — 4-bit fine-tuning
- “Self-Instruct” (Wang et al., 2022) — Synthetic data
- “Retrieval-Augmented Generation” (Lewis et al., 2020)
- “Chain-of-Thought Prompting” (Wei et al., 2022)
- “Judging LLM-as-a-Judge” (Zheng et al., 2023)
- “Training Language Models to Follow Instructions” (Ouyang et al., 2022) — InstructGPT

10.2 Must-Watch Videos

- Andrej Karpathy — “Intro to Large Language Models” (1hr)
- Andrej Karpathy — “Let’s build GPT from scratch” (2hr)
- 3Blue1Brown — “But what is a GPT?” series
- Stanford CS224N lectures (free on YouTube)

10.3 Must-Read Blogs

- Jay Alammar — “The Illustrated Transformer” and “The Illustrated Word2Vec”
- Sebastian Raschka — “Practical Tips for Finetuning LLMs”
- Hamel Husain — “Your AI Product Needs Evals”
- Chip Huyen — “Building LLM Applications for Production”
- Lilian Weng — “The Transformer Family” (lilianweng.github.io)

10.4 Hands-On Tutorials

- Hugging Face NLP Course (huggingface.co/learn/nlp-course)
- Hugging Face PEFT documentation
- Hugging Face SFTTrainer tutorial
- LangChain RAG tutorial
- Anthropic Prompt Engineering Guide

10.5 Books (Physical Copies Recommended)

Book	Author	Why Read It
Build a Large Language Model (From Scratch)	Sebastian Raschka	Best hands-on LLM internals. Code-first.
Designing Machine Learning Systems	Chip Huyen	Production ML, evaluation, pipelines. Great for interviews.
NLP with Transformers	Tunstall et al. (O'Reilly)	Comprehensive HF ecosystem guide.
Hands-On Large Language Models	Alammar & Grootendorst	Visual explanations, practical focus.
Deep Learning	Goodfellow et al.	Theory bible. Dense but authoritative.

Top 2 picks for physical copies: Sebastian Raschka's "Build a Large Language Model (From Scratch)" for deep understanding, and Chip Huyen's "Designing Machine Learning Systems" for interview prep.